

Reflective Code Loading

Adversaries may reflectively load code into a process in order to conceal the execution of malicious payloads. Reflective loading involves allocating then executing payloads directly within the memory of the process, vice creating a thread or process backed by a file path on disk (e.g., [Shared Modules](#)).

Reflectively loaded payloads may be compiled binaries, anonymous files (only present in RAM), or just snubs of fileless executable code (ex: position-independent shellcode).^{[1][2][3][4][5]} For example, the `Assembly.Load()` method executed by [PowerShell](#) may be abused to load raw code into the running process.^[6]

Reflective code injection is very similar to [Process Injection](#) except that the "injection" loads code into the processes' own memory instead of that of a separate process. Reflective loading may evade process-based detections since the execution of the arbitrary code may be masked within a legitimate or otherwise benign process. Reflectively loading payloads directly into memory may also avoid creating files or other artifacts on disk, while also enabling malware to keep these payloads encrypted (or otherwise obfuscated) until execution.^{[3][4][7][8]}

ID: T1620

Sub-techniques: No sub-techniques

① **Tactic:** [Stealth](#)

① **Platforms:** Linux, Windows, macOS

Contributors: Jiraput Thamsongkrah; Joas Antonio dos Santos, @C0d3Cr4zy, Inmetrics; João Paulo de A. Filho, @Hug1nN_; Lior Ribak, SentinelOne; Rex Guo, @Xiaofei_REX, Confluera; Shlomi Salem, SentinelOne

Version: 2.0

Created: 05 October 2021

Last Modified: 12 May 2026

[Version Permalink](#)

Procedure Examples

ID	Name	Description
C0057	3CX Supply Chain Attack	During the 3CX Supply Chain Attack , AppleJeus leverages the publicly available open-source project DAVESHELL to convert PE-COFF files to position-independent code to reflectively load the payload into memory. [9] [10]
S1081	BADHATCH	BADHATCH can copy a large byte array of 64-bit shellcode into process memory and execute it with a call to <code>CreateThread</code> . [11]
S9011	BRUSHFIRE	BRUSHFIRE has executed its commands within memory and is not saved on disk. [12] [13]
S1063	Brute Ratel C4	Brute Ratel C4 has used reflective loading to execute malicious DLLs. [14]
S0154	Cobalt Strike	Cobalt Strike 's <code>execute-assembly</code> command can run a .NET executable within the memory of a sacrificial process by loading the CLR. [15]
S0625	Cuba	Cuba loaded the payload into memory using PowerShell. [16]
S0695	Donut	Donut can generate code modules that enable in-memory execution of VBScript, JScript, EXE, DLL, and dotNET payloads. [17]
S0367	Emotet	Emotet has reflectively loaded payloads into memory. [18]
G0046	FIN7	FIN7 has loaded a .NET assembly into the current execution context via <code>Reflection.Assembly::Load</code> . [19]
S0661	FoggyWeb	FoggyWeb 's loader has reflectively loaded .NET-based assembly/payloads into memory. [20]
S9033	Fooder	Fooder has reflectively loaded a payload into memory. [21]
G0047	Gamaredon Group	Gamaredon Group has used an obfuscated PowerShell script that used <code>System.Reflection.Assembly</code> to gather and send victim information to the C2. [22]
S0666	Gelsemium	Gelsemium can use custom shellcode to map embedded DLLs into memory. [23]
S1022	IceApple	IceApple can use reflective code loading to load .NET assemblies into <code>MSExchangeOWAAppPool</code> on targeted Exchange servers. [24]
G0094	Kimsuky	Kimsuky has used the Invoke-Mimikatz PowerShell script to reflectively load a Mimikatz credential stealing DLL into memory. [25] Kimsuky has also used reflective loading through .NET assembly using <code>[System.Reflection.Assembly]::Load</code> . [26]
G0032	Lazarus Group	Lazarus Group has changed memory protection permissions then overwritten in memory DLL function code with shellcode, which was later executed via KernelCallbackTable hijacking. Lazarus Group has also used shellcode within macros to decrypt and manually map DLLs into memory at runtime. [27] [28]
S0681	Lizar	Lizar has used the Reflective DLL injection module from Github to inject itself into a process's memory. [29]
S0447	Lokibot	Lokibot has reflectively loaded the decoded DLL into memory. [30]
S1213	Lumma Stealer	Lumma Stealer has used reflective loading techniques to load content into memory during execution. [31] [32]
S1143	LunarLoader	LunarLoader can use reflective loading to decrypt and run malicious executables in a new thread. [33]
S1059	metaMain	metaMain has reflectively loaded a DLL to read, decrypt, and load an orchestrator file. [34]

ID	Name	Description
S0194	PowerSploit	PowerSploit reflectively loads a Windows PE file into a process. ^{[41][42]}
S1085	Sardonic	Sardonic has a plugin system that can load specially made DLLs into memory and execute their functions. ^{[43][44]}
C0058	SharePoint ToolShell Exploitation	During SharePoint ToolShell Exploitation , threat actors reflectively loaded payloads using <code>System.Reflection.Assembly.Load</code> . ^{[45][46][47][48][49]}
S0692	SILENTTRINITY	SILENTTRINITY can run a .NET executable within the memory of a sacrificial process by loading the CLR. ^[50]
S9001	SystemBC	SystemBC has downloaded a text file into memory and set the area of memory via the VirtualProtect call. Then, SystemBC has executed the file via the CreateThread call. ^[51]
S0595	ThiefQuest	ThiefQuest uses various API functions such as <code>NSCreateObjectFileImageFromMemory</code> to load and link in-memory payloads. ^[52]
S0022	Uroburos	Uroburos has the ability to load new modules directly into memory using its <code>Load Modules Mem</code> command. ^[53]
S0689	WhisperGate	WhisperGate 's downloader can reverse its third stage file bytes and reflectively load the file as a .NET assembly. ^[54]

Mitigations

This type of attack technique cannot be easily mitigated with preventive controls since it is based on the abuse of system features.

Detection Strategy

ID	Name	Analytic ID	Analytic Description
DET0300	Detection Strategy for Reflective Code Loading	AN0838	Detect anomalous chains of memory allocation and execution inside the same process (e.g., VirtualAlloc → memcpy → VirtualProtect → CreateThread). Unlike process injection, reflective code loading does not perform cross-process memory writes — the suspicious activity occurs entirely within the process's own PID context.
		AN0839	Monitor for in-process mmap + mprotect + execve/execveat activity where memory permissions are changed from writable to executable inside the same process without a corresponding ELF on disk.
		AN0840	Suspicious calls to dlopen(), dlsym(), or mmap with RWX flags in processes that do not typically perform dynamic module loading. Monitor anonymous memory regions executed by user processes.

References

1. [The Wover. \(2019, May 9\). Donut - Injecting .NET Assemblies as Shellcode. Retrieved October 4, 2021.](#)
2. [Bunce, D. \(2019, October 31\). Building A Custom Tool For Shellcode Analysis. Retrieved October 4, 2021.](#)
3. [Stuart. \(2018, March 31\). In-Memory-Only ELF Execution \(Without tmpfs\). Retrieved October 4, 2021.](#)
4. [0x00pico. \(2017, September 25\). Super-Stealthy Droppers. Retrieved October 4, 2021.](#)
5. [Kirk, N. \(2018, June 18\). Bring Your Own Land \(BYOL\) – A Novel Red Teaming Technique. Retrieved October 4, 2021.](#)
6. [Microsoft. \(n.d.\). Assembly.Load Method. Retrieved February 9, 2024.](#)
7. [Sanmillan, I. \(2019, November 18\). ACBackdoor: Analysis of a New Multiplatform Backdoor. Retrieved October 4, 2021.](#)
8. [Landry, J. \(2016, April 21\). Teaching an old RAT new tricks. Retrieved October 4, 2021.](#)
9. [Jeff Johnson, Fred Plan, Adrian Sanchez, Renato Fontana, Jake Nicastro, Dimiter Andonov, Marius Fodoreanu, Daniel Scott. \(2023, April 20\). 3CX Software Supply Chain Compromise Initiated by a Prior Software Supply Chain Compromise; Suspected North Korean Actor Responsible. Retrieved August 25, 2025.](#)
10. [Nick Landers \(monoxgas\). \(2022, June 18\). GitHub monoxgas sRDI \(DAVESHELL\). Retrieved October 1, 2025.](#)
11. [Savelesky, K., et al. \(2019, July 23\). ABADBABE 8BADFOOD: Discovering BADHATCH and a Detailed Look at FIN8's Tooling. Retrieved September 8, 2021.](#)
12. [John Wolfram, Michael Edie, Jacob Thompson, Matt Lin, Josh Murchie. \(2025, April 3\). Suspected China-Nexus Threat Actor Actively Exploiting Critical Ivanti Connect Secure Vulnerability \(CVE-2025-22457\). Retrieved April 13, 2026.](#)
13. [Sila Ozeren Hacioglu. \(2025, May 5\). UNC5221's Latest Exploit: Weaponizing CVE-2025-22457 in Ivanti Connect Secure. Retrieved April 13, 2026.](#)
14. [Chell, D. PART 3: How I Met Your Beacon – Brute Ratel. Retrieved February 6, 2023.](#)
15. [Strategic Cyber LLC. \(2020, November 5\). Cobalt Strike: Advanced Threat Tactics for Penetration Testers. Retrieved April 13, 2021.](#)
16. [Roccio, T., et al. \(2021, April\). Technical Analysis of Cuba Ransomware. Retrieved June 18, 2021.](#)
17. [TheWover. \(2019, May 9\). donut. Retrieved March 25, 2022.](#)
18. [Binary Defense. \(n.d.\). Emotet Evolves With new Wi-Fi Spreader. Retrieved September 8, 2023.](#)
19. [Gemini Advisory. \(2022, January 13\). FIN7 Uses Flash Drives to Spread Remote Access Trojan. Retrieved May 14, 2025.](#)
20. [Ramin Nafisi. \(2021, September 27\). FoggyWeb: Targeted NOBELIUM malware leads to persistent backdoor. Retrieved October 4, 2021.](#)
21. [ESET Research. \(2025, December 2\). MuddyWater: Snakes by the riverbank. Retrieved February 17, 2026.](#)
22. [Threat Hunter Team, Symantec and Carbon Black. \(2025, April 10\). Shuckworm Targets Foreign Military Mission Based in Ukraine. Retrieved July 23, 2025.](#)
23. [Dupuy, T. and Faou, M. \(2021, June\). Gelsemium. Retrieved November 30, 2021.](#)
24. [CrowdStrike. \(2022, May\). ICEAPPLE: A NOVEL INTERNET](#)
28. [Pradhan, A. \(2022, February 8\). LolZarus: Lazarus Group Incorporating Lolbins into Campaigns. Retrieved March 22, 2022.](#)
29. [Bourhis, P., Sekoia TDR. \(2024, February 1\). Unveiling the intricacies of DiceLoader. Retrieved May 14, 2025.](#)
30. [Muhammad, I., Unterbrink, H.. \(2021, January 6\). A Deep Dive into Lokibot Infection Chain. Retrieved August 31, 2021.](#)
31. [Leandro Fróes, Netskope. \(2025, January 23\). Lumma Stealer: Fake CAPTCHAs & New Techniques to Evade Detection. Retrieved March 22, 2025.](#)
32. [Cara Lin, Fortinet. \(2024, January 8\). Deceptive Cracked Software Spreads Lumma Variant on YouTube. Retrieved March 22, 2025.](#)
33. [Jurčacko, F. \(2024, May 15\). To the Moon and back\(doors\): Lunar landing in diplomatic missions. Retrieved June 26, 2024.](#)
34. [Ehrlich, A., et al. \(2022, September\). THE MYSTERY OF METADOR | AN UNATTRIBUTED THREAT HIDING IN TELCOS, ISPS, AND UNIVERSITIES. Retrieved January 23, 2023.](#)
35. [Daniel Stepanic & Salim Bitam. \(2024, February 23\). PIKABOT, I choose you!. Retrieved July 12, 2024.](#)
36. [Alexandre Cote Cyr. \(2022, March 23\). Mustang Panda's Hodur: Old tricks, new Korplug variant. Retrieved September 9, 2025.](#)
37. [EclecticIQ Threat Research Team. \(2023, February 2\). Mustang Panda APT Group Uses European Commission-Themed Lure to Deliver PlugX Malware. Retrieved September 9, 2025.](#)
38. [Patrick Whitsell. \(2025, August 25\). Deception in Depth: PRC-Nexus Espionage Campaign Hijacks Web Traffic to Target Diplomats. Retrieved September 9, 2025.](#)
39. [Secureworks Counter Threat Unit Research Team. \(2022, April 27\). BRONZE PRESIDENT Targets Russian Speakers with Updated PlugX. Retrieved September 9, 2025.](#)
40. [Secureworks Counter Threat Unit Research Team. \(2022, September 8\). BRONZE PRESIDENT Targets Government Officials. Retrieved September 9, 2025.](#)
41. [PowerShellMafia. \(2012, May 26\). PowerSploit - A PowerShell Post-Exploitation Framework. Retrieved February 6, 2018.](#)
42. [PowerSploit. \(n.d.\). PowerSploit. Retrieved February 6, 2018.](#)
43. [Budaca, E., et al. \(2021, August 25\). FIN8 Threat Actor Goes Agile with New Sardonic Backdoor. Retrieved August 9, 2023.](#)
44. [Symantec Threat Hunter Team. \(2023, July 18\). FIN8 Uses Revamped Sardonic Backdoor to Deliver Noberus Ransomware. Retrieved August 9, 2023.](#)
45. [Microsoft Threat Intelligence. \(2025, July 22\). Disrupting active exploitation of on-premises SharePoint vulnerabilities. Retrieved October 15, 2025.](#)
46. [Eye Security. \(2025, July 19\). SharePoint Under Siege: ToolShell Exploit \(CVE-2025-49706 & CVE-2025-49704\). Retrieved October 15, 2025.](#)
47. [Trend Micro Research. \(2022, July 22\). Proactive Security Insights for SharePoint Attacks \(CVE-2025-53770 and CVE-2025-53771\). Retrieved October 15, 2025.](#)
48. [Kenin, S. et al. \(2025, July 21\). SharePoint ToolShell | Zero-Day Exploited in-the-Wild Targets Enterprise Servers. Retrieved October 15, 2025.](#)
49. [Unit 42. \(2025, July 31\). Active Exploitation of Microsoft SharePoint Vulnerabilities: Threat Brief \(Updated\). Retrieved October 15, 2025.](#)

27. [Saini, A. and Hossein, J. \(2022, January 27\). North Korea's Lazarus APT leverages Windows Update client, GitHub in latest campaign. Retrieved January 27, 2022.](#)

54. [Insikt Group. \(2020, January 28\). WhisperGate Malware Corrupts Computers in Ukraine. Retrieved September 16, 2024.](#)